

Agent sinh pipeline ETL và dashboard từ CSV

NỘI DUNG

- 1

Tổng quan dự án
- 2

Thực trạng
- 3

Giải pháp
- 4

Luồng người dùng
- 5

Yêu cầu chức năng
- 6

Tech stack
- 7

Giới hạn bản đầu và hướng mở rộng

I Tổng quan dự án

Web app nhận các file CSV thô cùng một yêu cầu phân tích viết bằng ngôn ngữ tự nhiên, và trả ra ba thứ: một warehouse đã dựng, một bộ dashboard trên Metabase, và toàn bộ source code pipeline dưới dạng dbt project chạy được độc lập.

Hệ thống tự động hóa phần có khuôn mẫu trong việc tiếp nhận một nguồn dữ liệu mới: khảo sát dữ liệu, đề xuất mô hình, viết pipeline, sinh test, tối ưu, dựng báo cáo.

BA ĐIỂM PHÂN BIỆT

Mã được *kiểm chứng bằng cách chạy thật* và đối chiếu kết quả, không chỉ được sinh ra.

Optimizer chỉ giữ lại rewrite vừa nhanh hơn vừa cho kết quả tương đương, kiểm chứng trên bộ dữ liệu thử.

Hai cổng duyệt bắt buộc; hệ thống không tác động vào môi trường production của người dùng.

2 Thực trạng

Mỗi khi tiếp nhận một nguồn dữ liệu mới, đội data lập lại gần như nguyên vẹn một chuỗi công việc đã làm nhiều lần: khảo sát dữ liệu, thiết kế mô hình, viết pipeline, viết test, tối ưu, dựng báo cáo. Các công cụ sinh SQL bằng LLM hiện nay tập trung vào việc trả lời câu hỏi trên schema có sẵn; dự án này nhắm vào đoạn trước đó.

Nhóm đã xây dựng OULAD Student Data Warehouse — 7 file CSV, 32.000 sinh viên, 10,6 triệu bản ghi tương tác, kiến trúc bronze → silver → star schema → dbt marts → Metabase — trong 7 tuần. Con số này gồm cả thời gian học công nghệ và viết báo cáo, nên không thể coi toàn bộ là phần tự động hóa được. Điều rút ra là phân loại tính chất công việc:

CÔNG VIỆC	TÍNH CHẤT
Khảo sát dữ liệu, suy ra kiểu cột và quan hệ	Có khuôn mẫu, lặp lại
Thiết kế star schema, chọn grain, định nghĩa chỉ số	Cần phán đoán nghiệp vụ
Viết dbt model, viết test, dựng DAG	Có khuôn mẫu, lặp lại
Tối ưu query	Có khuôn mẫu, đo được bằng số

Ba nhóm có khuôn mẫu là dư địa tự động hóa. Nhóm cần phán đoán nghiệp vụ là lý do hệ thống phải có cổng duyệt của con người.

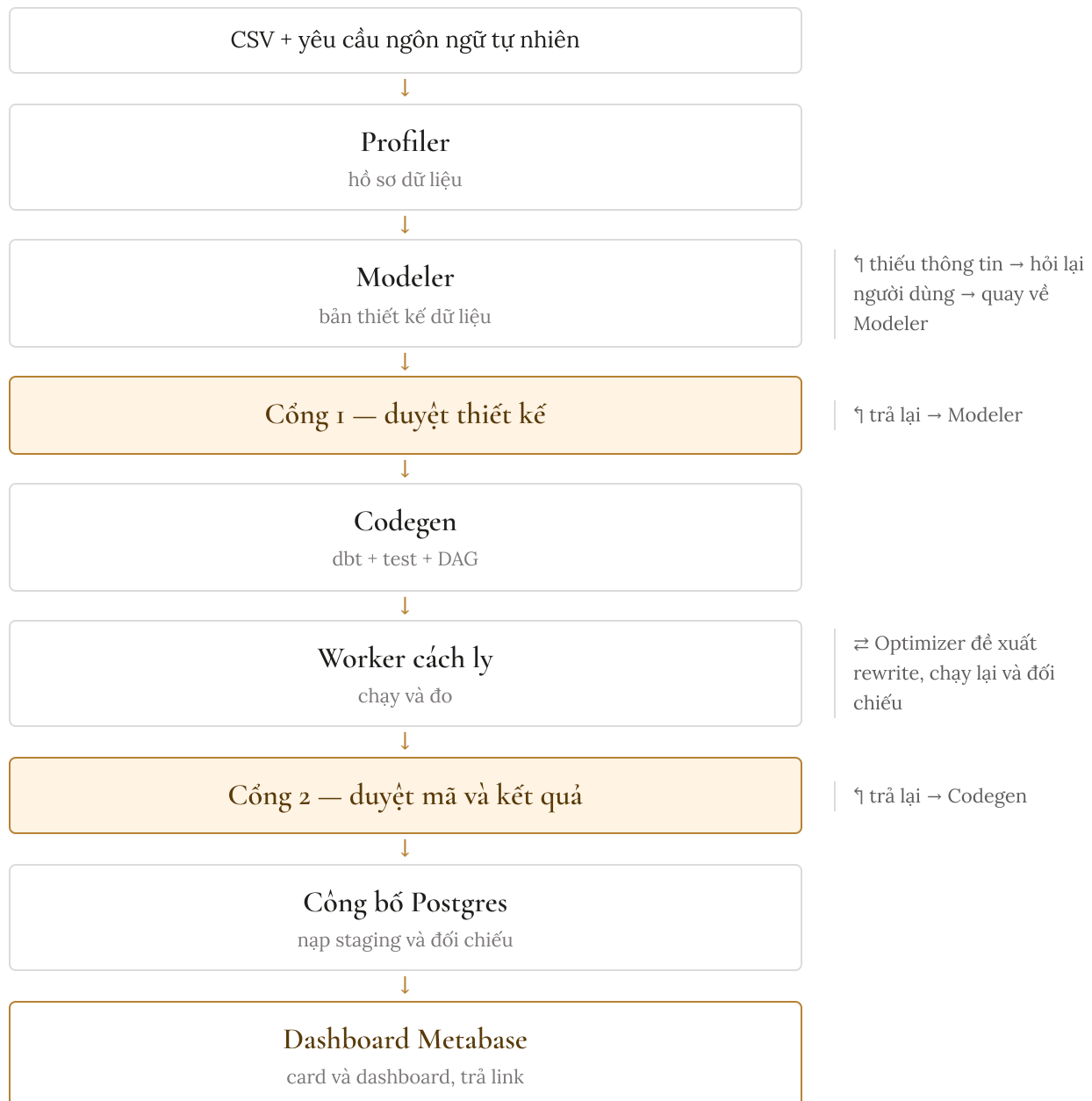
BỐN VẤN ĐỀ

- 1 Thời gian khởi tạo dài** — phần lớn dành cho mã khuôn mẫu.
- 2 Chất lượng phụ thuộc kinh nghiệm cá nhân** — chọn sai grain khiến toàn bộ mart phải làm lại.
- 3 Tối ưu bị hoãn** — chỉ làm khi dashboard đã chậm thấy rõ, lúc đó sửa tốn hơn.
- 4 Không có kiểm chứng tự động** — chạy không lỗi không có nghĩa kết quả đúng.

3 Giải pháp

Hệ multi-agent chạy sau giao diện web, xen giữa là hai cổng duyệt của con người và một vòng tối ưu có kiểm chứng.

SƠ ĐỒ LƯỚI



Phân vai

THÀNH PHẦN	LOẠI	NHIỆM VỤ
Profiler	Xác định	Thống kê, kiểm tra khóa ứng viên và quan hệ, kèm bằng chứng
Modeler	LLM	Diễn giải yêu cầu, đề xuất thiết kế, hỏi lại khi mơ hồ
Codegen	LLM	Sinh và sửa dbt model, test, cấu hình DAG
Optimizer	LLM + quy tắc	Đề xuất phương án rewrite SQL
Bộ thực thi và đối chiếu	Xác định	Chạy, đo, so sánh, quyết định giữ hay loại

Nguyên tắc: LLM đề xuất, công cụ xác định phán xét. Không thành phần LLM nào tự chấm kết quả của chính nó.

Cách ly thực thi

DuckDB là engine thực thi, không tự tạo ranh giới bảo vệ: SQL do LLM sinh có thể đọc ghi file và tiêu thụ tài nguyên theo quyền của tiến trình chạy nó; DAG Python và dbt macro cũng vậy. Vì thế mã sinh ra không chạy trong tiến trình backend.

- FastAPI chỉ tiếp nhận yêu cầu và quản lý trạng thái. Worker riêng chạy dbt và DuckDB, mỗi lượt một thư mục riêng, có giới hạn RAM, CPU, thời gian và dung lượng.
- Worker không giữ thông tin kết nối tới Postgres phục vụ. Bước công bố do publisher thực hiện sau khi duyệt.
- DAG sinh từ template cố định, chỉ thay cấu hình và danh sách tác vụ; không nạp mã Python tùy ý vào Airflow đang vận hành.

Ba môi trường tách biệt: **sandbox xây dựng** chạy mã đang sinh hoặc sửa, **môi trường chạy pipeline đã duyệt** thực thi đúng phiên bản đã duyệt và vẫn dùng DuckDB, **Postgres phục vụ** chỉ nhận mart đã đạt kiểm tra.

Hai cổng duyệt

Cổng 1 duyệt một **bản thiết kế dữ liệu**, không chỉ sơ đồ schema:

NỘI DUNG	VÍ DỤ
Grain	Một dòng fact ứng với một dòng sản phẩm trong đơn
Khóa và quan hệ	Khóa ghép, khóa nghiệp vụ, quan hệ một-nhiều
Định nghĩa chỉ số	Doanh thu tính trước hay sau hoàn tiền
Quy tắc làm sạch	Dòng thiếu khóa bị cách ly hay làm dừng pipeline
Quy tắc cập nhật	Lô mới thay thế toàn bộ hay bổ sung
Yêu cầu dashboard	Chỉ số, chiều phân tích, bộ lọc, loại card

Khi hồ sơ thống kê không đủ để quyết định, Modeler hỏi lại thay vì đoán: cột `amount` không cho biết đó là đơn giá, thành tiền hay số tiền đã thanh toán.

Cổng 2 duyệt diff mã, kết quả test, bảng so sánh hiệu năng và cấu hình dashboard.

Quan sát không phải ràng buộc

Thống kê trên lô đầu không đủ thành quy tắc chặn: một cột tình cờ không trùng không có nghĩa nó là khóa, và một trạng thái mới có thể xuất hiện hợp lệ ở lô sau. Quan sát thống kê chỉ dùng để đề xuất kèm bằng chứng; chỉ ràng buộc đã được duyệt mới sinh test bắt buộc; giả định chưa xác nhận thì hỏi lại hoặc cảnh báo.

Test áp đúng tầng: bronze chấp nhận bản ghi lỗi, silver xử lý hoặc cách ly, mart phải đạt ràng buộc đã chốt. Chất lượng bộ test đo bằng tỉ lệ pass trên dữ liệu sạch và tỉ lệ bắt được lỗi khi tiêm lỗi có chủ đích.

Vòng tối ưu

Phạm vi bản đầu là **rewrite SQL trên DuckDB**; index ở Postgres và phân vùng file là hướng mở rộng. sqlglot parse và biến đổi SQL đã compile; kế hoạch thực thi và số đo lấy từ profiling của DuckDB.

Hai phép kiểm chứng tách biệt: đầu ra so với đáp án chuẩn trả lời *pipeline có đúng không*; sau rewrite so với trước rewrite trả lời *rewrite có giữ nguyên kết quả không*. Giữ nguyên một kết quả sai không làm pipeline trở thành đúng.

TƯƠNG ĐƯƠNG

Schema đầu ra khớp; so sánh theo multiset có đếm số lần xuất hiện từng dòng, không chỉ row count; quy tắc rõ cho NULL, số thực và timestamp; giá trị phụ thuộc thời gian chạy được cố định qua tham số. Kết luận giới hạn ở bộ dữ liệu kiểm thử.

NHANH HƠN

Cùng dữ liệu, phiên bản engine và cấu hình tài nguyên; chạy lặp lấy median; cải thiện phải lớn hơn biên độ nhiễu đo; đo cả từng model và toàn pipeline.

Vòng lặp có ngân sách: số ứng viên tối đa, số lần sửa lỗi tối đa, thời gian tối đa, điều kiện dừng. Khi mọi rewrite đều bị loại, giữ bản hợp lệ tốt nhất. Query vốn đã tốt và không cần rewrite là kết quả hợp lệ.

Đây là tối ưu **thời gian dựng dữ liệu trên DuckDB**, không đồng nghĩa dashboard trên Postgres nhanh hơn.

Truy xuất mẫu

Kho mẫu lưu trong Qdrant, gồm cặp mô tả yêu cầu kèm dbt model tương ứng, thu từ các dự án dbt mã nguồn mở và phần tập chuẩn dành riêng. Trước khi sinh mã, hệ thống truy xuất vài mẫu gần nhất đưa vào ngữ cảnh Codegen.

Đo lường

Bộ eval chạy offline, tách khỏi web app, nạp tập chuẩn cố định gồm OULAD và hai bộ công khai khác; mỗi tác vụ có bản thiết kế chuẩn và bảng kết quả chuẩn do nhóm viết tay.

Hai baseline: **nội bộ** là output trước khi optimizer can thiệp, đo tác dụng riêng của bước tối ưu; **con người** là pipeline OULAD viết tay, chỉ có nghĩa khi chuẩn hóa engine, phần cứng và phạm vi đo, nếu không thì ghi là so sánh giữa hai hệ thống.

Số dòng quét và bộ nhớ đỉnh là **chỉ số tài nguyên**, không quy đổi thành tiền. Chi phí token và thời gian tìm rewrite cũng được ghi lại.

NHÓM CHỈ SỐ	NỘI DUNG ĐO
Hợp lệ kỹ thuật	dbt compile, DAG import và chạy được trong môi trường thử
Đúng dữ liệu	Tỉ lệ bảng kết quả khớp đáp án chuẩn
Chất lượng thiết kế	Chấm grain, khóa và quan hệ; chấp nhận nhiều thiết kế hợp lệ
Chất lượng test	Pass trên dữ liệu sạch; bắt được lỗi khi tiêm lỗi có chủ đích
Hiệu năng	Cải thiện median thời gian chạy và số dòng quét so với baseline nội bộ
Rewrite bị loại	Phân loại: sai dữ liệu, lỗi chạy, không nhanh hơn, quá thời gian
Khả dụng	Số dòng mã Reviewer phải sửa, số vòng sửa, tỉ lệ tác vụ hoàn tất
Ablation	Có và không có retrieval, optimizer, sinh test

Tập mẫu retrieval tách khỏi tập đo, tách cả biến thể cùng tác vụ và cùng dataset, để tránh rò rỉ đáp án.

4 Luồng người dùng

Engineer nạp dữ liệu, mô tả yêu cầu, duyệt bản thiết kế và sửa mã. **Reviewer** duyệt mã cùng bằng chứng trước khi công bố. Chế độ nhóm yêu cầu Reviewer khác Engineer; chế độ cá nhân cho phép một người làm cả hai, và nhật ký ghi rõ là không có review độc lập.

I Nạp dữ liệu

Engineer tải CSV và gõ yêu cầu. Hệ thống đoán encoding, dấu phân cách, ký hiệu null và định dạng ngày, hiển thị vài dòng đầu để xác nhận. File gốc và cấu hình đọc được lưu nguyên vẹn.

2 Lập hồ sơ

Profiler quét bằng DuckDB: kiểu suy ra, tỉ lệ null, số giá trị phân biệt, min và max, mẫu giá trị; dò khóa chính ứng viên kể cả khóa ghép, và quan hệ ứng viên kèm bằng chứng gồm tỉ lệ khớp khóa, bản ghi không tìm thấy cha, khả năng join làm tăng số dòng. Chỉ hồ sơ thống kê được gửi vào LLM — cách này giảm đáng kể dữ liệu rời khỏi hệ thống nhưng không bảo đảm tuyệt đối, vì tên cột và giá trị hiếm vẫn có thể mang thông tin nhạy cảm.

3 Đề xuất thiết kế

Modeler đề xuất bản thiết kế dữ liệu kèm lý giải ngắn, hỏi lại khi thông tin không đủ.

4 Cổng 1

Engineer duyệt, sửa trực tiếp, hoặc trả lại. Sửa một bản thiết kế tốn vài phút; sửa hai mươi file dbt đã sinh tốn cả buổi.

5 Sinh mã và chạy thử

Codegen sinh dbt model ba tầng, test và cấu hình DAG. **Bronze** giữ biểu diễn nguồn, không ép kiểu nghiệp vụ, để mã như `00123` không thành số; mỗi dòng mang `batch_id`, tên file, định danh dòng, thời điểm nạp. **Silver** chuẩn hóa kiểu, xử lý null, khử trùng lặp; dòng lỗi bị cách ly kèm lý do và báo cáo số lượng. **Mart** dựng fact và dimension theo thiết kế đã duyệt. Toàn bộ chạy trong worker cách ly, ghi lại thời gian chạy, số dòng quét, bộ nhớ đỉnh và kết quả test.

6 Tối ưu

Optimizer xếp hạng query theo thời gian chạy, đề xuất rewrite trong ngân sách đã định, mỗi đề xuất được chạy lại và đối chiếu theo tiêu chí tương đương ở mục 3. Đề xuất bị loại được ghi nhật ký kèm lý do.

7 Cổng 2

Reviewer xem diff mã, kết quả test, so sánh hiệu năng và cấu hình dashboard. Kết quả duyệt gắn với một phiên bản cụ thể gồm thiết kế, mã và test, lô dữ liệu, lượt chạy và báo cáo. Nếu mã bị sửa sau khi duyệt, báo cáo cũ mất hiệu lực: phải chạy lại kiểm tra và duyệt lại.

8 Công bố sang Postgres

Nạp mart vào staging của Postgres → đối chiếu dữ liệu → công bố phiên bản. Nếu bước nạp hỏng giữa chừng, dashboard vẫn phục vụ phiên bản đã công bố trước đó.

9 Tạo dashboard trên Metabase

Bước cuối của luồng. Đồng bộ metadata với Metabase → tạo hoặc cập nhật card → gom card vào dashboard → kiểm tra truy vấn từng card trước khi trả link cho Engineer. Ba loại card: KPI dạng số, biểu đồ đường, biểu đồ cột. ID card và dashboard được lưu để retry không tạo bản trùng và để chạy lại riêng bước này mà không phải nạp lại dữ liệu.

Chạy lại theo lịch

Pipeline đã duyệt được đăng ký thành DAG trong Airflow. Bản đầu chỉ hỗ trợ **snapshot thay thế toàn bộ**; append và upsert theo khóa là hướng mở rộng.

TÌNH HUỐNG	HÀNH VI
Một lô cần nhiều file	Chỉ chạy khi bộ file đủ và được đánh dấu sẵn sàng
Cùng lô chạy hai lần	Thay thế, không nhân đôi dữ liệu
Lô mới đổi schema	Dừng và hỏi Engineer
Hai lượt chạy trùng thời điểm	Khóa theo pipeline, đưa vào hàng đợi
Bước công bố thất bại	Retry được, không tạo dữ liệu hay dashboard trùng

Mỗi lượt chạy lại vẫn thực thi bộ test đã duyệt; test fail thì dừng và báo, không công bố dữ liệu sai.

Bàn giao mã nguồn

Ba đường ra: tải ZIP, mở Pull Request vào repo người dùng chỉ định, hoặc sao chép từng file trong trình soạn thảo. ZIP và PR luôn chứa cùng phiên bản đã duyệt; hệ thống không tự merge.

THÀNH PHẦN	VAI TRÒ
<code>models/</code> , <code>dbt_project.yml</code> , <code>profiles.yml</code> mẫu	Pipeline dbt
<code>schema.yml</code> , SQL test tùy chỉnh, macro	Bộ kiểm thử đầy đủ
Loader CSV và manifest nguồn	Tái tạo bước nạp dữ liệu
Script chuyển mart sang Postgres	Hoàn tất pipeline như trên nền tảng
Cấu hình dashboard và script dựng lại	Tái tạo đầu ra Metabase
Dockerfile, Compose, dependency cố định phiên bản	Dựng môi trường
File DAG	Chạy định kỳ trên Airflow của người dùng
Manifest phiên bản và báo cáo kiểm chứng	Xác định đúng bộ mã đã kiểm chứng

Gói xuất giữ đúng kiến trúc DuckDB → Postgres → Metabase đã kiểm thử; không chuyển mã sang warehouse khác, vì transpile không xử lý được adapter, macro và materialization.

Tiêu chí nghiệm thu: một máy sạch tải ZIP, điền cấu hình, cung cấp CSV, và dựng lại được mart cùng dashboard mà không gọi API của nền tảng. Sau bàn giao, sửa trong trình soạn thảo tạo phiên bản mới phải test lại; sửa ở máy người dùng là nhánh độc lập, hệ thống không đồng bộ ngược.

5 Yêu cầu chức năng

Tất cả đều bắt buộc trong bản đầu. Cột nghiệm thu là điều kiện để đánh dấu hoàn thành.

MÃ	CHỨC NĂNG	AI THỰC HIỆN	TIÊU CHÍ NGHIỆM THU
FR-01	Tải CSV và nhập yêu cầu	Engineer	Đọc đúng encoding và dấu phân cách, hiển thị preview, lưu file gốc cùng cấu hình đọc
FR-02	Lập hồ sơ dữ liệu	Hệ thống	Trả về thống kê từng cột; không gửi dữ liệu thô vào LLM
FR-03	Dò khóa và quan hệ giữa các file	Hệ thống	Mỗi ứng viên kèm tỉ lệ khớp khóa và cảnh báo join làm tăng số dòng
FR-04	Đề xuất bản thiết kế dữ liệu	Modeler	Đủ sáu nội dung ở mục 3, mỗi lựa chọn có lý giải
FR-05	Hỏi lại khi thiếu thông tin	Modeler	Nghiệp vụ hoặc grain mơ hồ thì hỏi trước khi chốt thiết kế
FR-06	Duyệt thiết kế (công 1)	Engineer	Không sinh mã khi chưa có xác nhận
FR-07	Sinh dbt model ba tầng	Codegen	dbt compile thành công; bronze giữ biểu diễn nguồn
FR-08	Sinh test từ ràng buộc đã duyệt	Codegen	Không sinh test bắt buộc từ quan sát chưa được duyệt
FR-09	Sinh file DAG từ template	Codegen	DAG import được; không nạp mã Python tùy ý vào Airflow
FR-10	Chạy pipeline trong worker cách ly	Hệ thống	Worker có giới hạn tài nguyên và không giữ thông tin kết nối warehouse
FR-11	Đo hiệu năng từng model	Hệ thống	Ghi thời gian chạy, số dòng quét, bộ nhớ đỉnh

MÃ	CHỨC NĂNG	AI THỰC HIỆN	TIÊU CHÍ NGHIỆM THU
FR-12	Đề xuất và kiểm chứng rewrite	Optimizer	Rewrite đổi kết quả bị loại; mọi rewrite bị loại thì giữ bản hợp lệ trước đó
FR-13	Duyệt mã và bằng chứng (cổng 2)	Reviewer	Không thể công bố phiên bản khác bằng phê duyệt của phiên bản cũ
FR-14	Công bố sang Postgres	Publisher	Nạp staging rồi đối chiếu; lỗi giữa chừng không phá phiên bản đang phục vụ
FR-15	Dựng card và dashboard Metabase	Hệ thống	Kiểm tra truy vấn từng card trước khi trả link; retry không tạo bản trùng
FR-16	Đăng ký DAG chạy lại theo lịch	Engineer	Cùng một lô chạy hai lần không nhân đôi dữ liệu
FR-17	Dừng khi lô mới đổi schema	Hệ thống	Không tự áp dụng thay đổi schema chưa được duyệt
FR-18	Xuất ZIP project độc lập	Engineer	Máy sạch dựng lại được mart và dashboard, không gọi API nền tảng
FR-19	Mở Pull Request	Engineer	PR chứa đúng phiên bản đã duyệt; hệ thống không tự merge
FR-20	Xác thực và phân quyền theo dự án	Hệ thống	Chỉ Reviewer có quyền duyệt; lịch sử duyệt gắn với dự án
FR-21	Khôi phục lượt đang chờ duyệt	Hệ thống	Backend khởi động lại vẫn tiếp tục đúng trạng thái

6 Tech stack

LAYER	TECHNOLOGY	PURPOSE
Backend Core	FastAPI · Python 3.11 · Pydantic v2	REST API bất đồng bộ, kiểm tra cấu trúc đầu ra của agent
Orchestration	LangGraph (StateGraph , checkpointer trên Postgres)	Luồng có vòng lặp và điểm dừng HITL, khôi phục sau khi restart
LLM & Embeddings	Google Gemini · gemini-3.5-flash-lite · gemini-embedding-001	Modeler, Codegen, đề xuất rewrite, tìm mẫu pipeline
Data Transform	dbt-core · dbt-duckdb · DuckDB	Model bronze–silver–mart, chạy và đo trong worker cách ly
SQL Analysis	sqlglot · DuckDB profiling	Parse và rewrite SQL; lấy kế hoạch và số đo thực thi
Pipeline Scheduler	Apache Airflow (LocalExecutor)	Chạy lại pipeline đã duyệt theo lịch
Relational DB	PostgreSQL · SQLAlchemy · Alembic	Warehouse phục vụ, metadata ứng dụng, bảng job, metadata Airflow và Metabase
Vector DB	Qdrant	Kho mẫu pipeline cho truy xuất few-shot
File Storage	Volume bền vững dùng chung giữa worker và Airflow	CSV gốc, manifest lô, mã và báo cáo từng phiên bản
Publishing & Export	psycpg (COPY) · GitHub API · ZIP	Nạp Postgres, mở Pull Request, đóng gói project
BI	Metabase · REST API	Card và dashboard sinh tự động
Frontend SPA	React 19 · Vite · TypeScript · CSS · Monaco editor	Giao diện Engineer và Reviewer, review diff mã
Evaluation	pytest · bộ đối chiếu SQL tự xây · golden dataset	Đúng dữ liệu, tương đương rewrite, benchmark offline

LAYER	TECHNOLOGY	PURPOSE
Deployment	Docker Compose · Vercel · phiên bản cố định	Frontend tĩnh trên Vercel; backend và các service dựng bằng Compose

Không dùng `latest` cho bất kỳ thành phần nào. Tổ hợp phiên bản tương thích của Airflow, dbt-core, dbt-duckdb và DuckDB được chốt bằng cách dựng thử trong tuần đầu rồi khóa lại.

Quyết định kiến trúc

- **Warehouse phục vụ là Postgres**, vì Metabase không có driver DuckDB chính thức. DuckDB là nơi biến đổi và đo đạc.
- **Một instance Postgres, nhiều database tách vai trò**: warehouse phân tích, metadata ứng dụng, metadata Airflow, application database của Metabase. Tách database không đồng nghĩa dựng thêm container.
- **LangGraph điều phối việc viết pipeline, Airflow điều phối việc chạy pipeline**. Hai thứ không thay thế nhau.
- **Worker nhận việc qua bảng job trong Postgres**, có khóa hàng, retry và phục hồi. Không thêm Redis hay Celery ở bản đầu.
- **Mỗi lượt chạy có workspace và file DuckDB riêng**, tránh nhiều worker cùng ghi một file.
- **Chỉ publisher có quyền ghi warehouse**. Worker chạy mã thử nghiệm không giữ quyền ghi và không giữ thông tin kết nối. Metabase đọc bằng tài khoản chỉ đọc.
- **Polling thay cho WebSocket**, vì trạng thái đã lưu bền vững nên backend khởi động lại vẫn khôi phục được lượt đang chờ duyệt.

Triển khai

Môi trường phát triển của nhóm là Docker Compose trên máy đơn, cũng là thứ nộp kèm mã nguồn. Bản triển khai phục vụ demo dùng một cấu hình tham chiếu duy nhất, chọn và đo thực tế trước khi chốt:

THÀNH PHẦN	NOI CHẠY
SPA React build bằng Vite	Vercel, dạng tĩnh
FastAPI và worker thực thi	Container có volume bền vững
Airflow webserver và scheduler	Container riêng, dùng chung volume với worker
Postgres	Dịch vụ Postgres có lưu trữ bền vững
Metabase	Container riêng

Vì frontend là SPA tĩnh, Vercel chỉ phục vụ file đã build và không chạy hàm serverless, nên giới hạn thời gian thực thi của Vercel không ảnh hưởng tới hệ thống.

Một ràng buộc phải kiểm tra khi chọn nền tảng: filesystem mặc định của Cloud Run nằm trong bộ nhớ và biến mất khi instance dừng, nên đường dẫn file trong backend không thể là nơi lưu lâu dài cho các lượt chạy lại của Airflow. Nền tảng nào được chọn cũng phải nêu rõ worker, scheduler, volume, kết nối mạng và cách phục hồi job.

7 Giới hạn bản đầu và hướng mở rộng

Những giới hạn dưới đây là lựa chọn có chủ đích để giữ phạm vi, không phải thiếu sót.

GIỚI HẠN BẢN ĐẦU	HƯỚNG MỞ RỘNG
Chỉ snapshot thay thế toàn bộ khi chạy lại	Append, upsert theo khóa, SCD Type 2, xử lý bản ghi đến muộn
Optimizer chỉ rewrite SQL trên DuckDB	Tạo index ở Postgres, bố trí phân vùng file
Chỉ đo thời gian dựng dữ liệu	Đo riêng tốc độ truy vấn BI trên Postgres
Gói xuất giữ nguyên kiến trúc DuckDB → Postgres	Chuyển mã sang warehouse khác
Không đồng bộ ngược thay đổi từ máy người dùng	Import lại hoặc đồng bộ Git hai chiều
Xác thực và phân quyền tối thiểu cho hai vai trò	Tổ chức, nhóm, mời thành viên, phân quyền chi tiết
Ba loại card cố định	Agent tự chọn loại biểu đồ theo dữ liệu

Tiêu chí nghiệm thu

Bốn tình huống bắt buộc phải xử lý đúng trước khi coi hệ thống là hoàn chỉnh:

TÌNH HUỐNG	KẾT QUẢ BẮT BUỘC
Rewrite chạy nhanh hơn nhưng đổi kết quả	Loại rewrite, giữ bản hợp lệ trước đó
Mã bị sửa sau khi đã test	Kết quả duyệt cũ mất hiệu lực, phải chạy lại và duyệt lại
Cùng một lô chạy hai lần	Không nhân đôi dữ liệu
Tải ZIP sang máy sạch	Dựng lại được mart và dashboard, không gọi API nền tảng

Các tình huống còn lại — lỗi giữa chừng khi nạp Postgres, retry API Metabase, lô mới đổi schema, hai lượt chạy trùng thời điểm — đã được quy định hành vi ở mục 4 và sẽ kiểm thử nếu còn thời gian.